Wrocław University of Technology

IT Applications:
Electronic media in business and commerce

Lecture 2:
REST

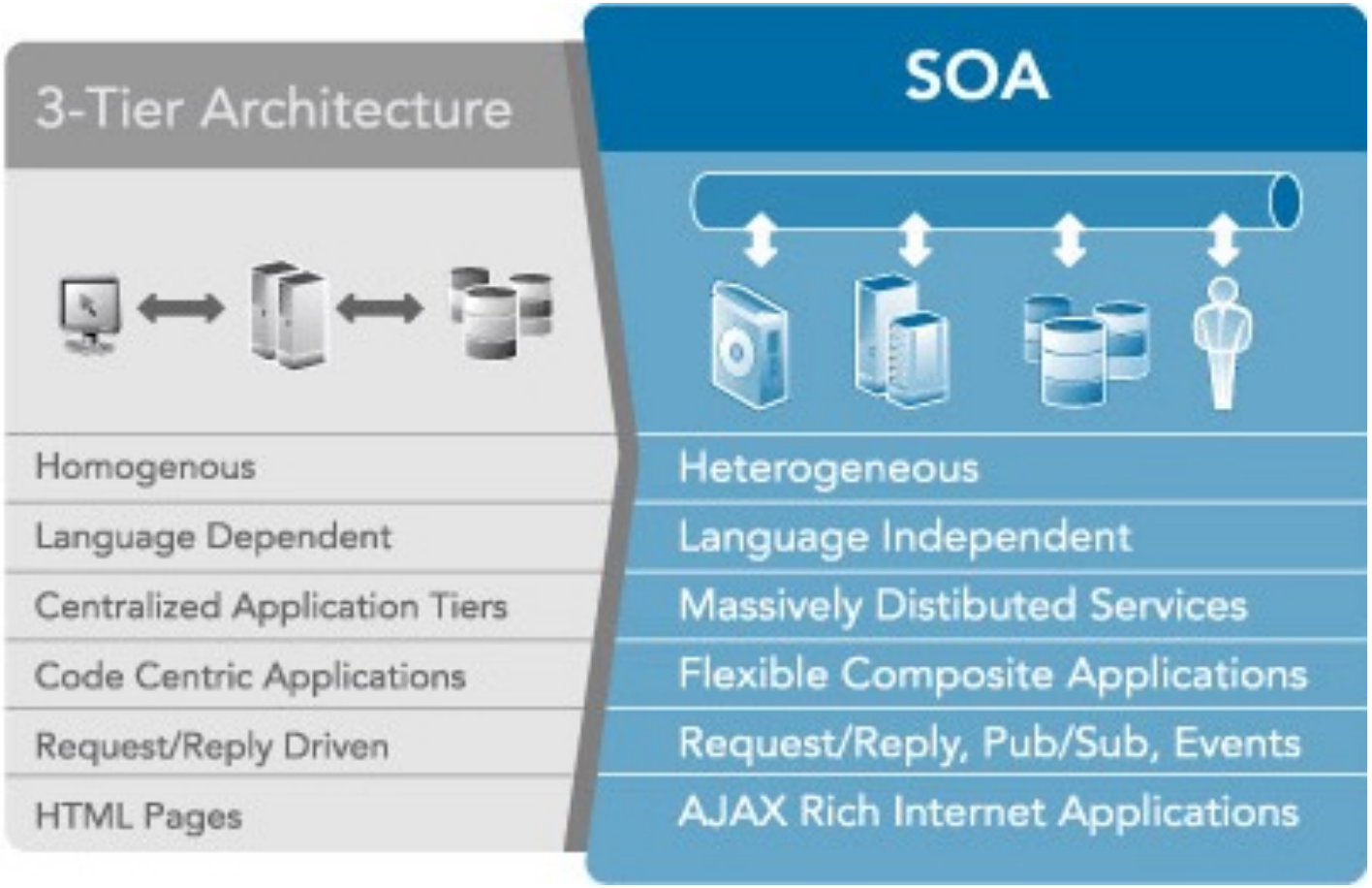
dr inż. Tomasz Walkowiak

Recap

- Reusable units, distributed system
- We need a kind of RPC
  - communication protocol
  - data exchange format
  - IDL – interface exchange
- Presented solutions
  - CORBA
    - Lack of security, firewall
    - Binary formats

SOA is an evolutionary step

- for architecture



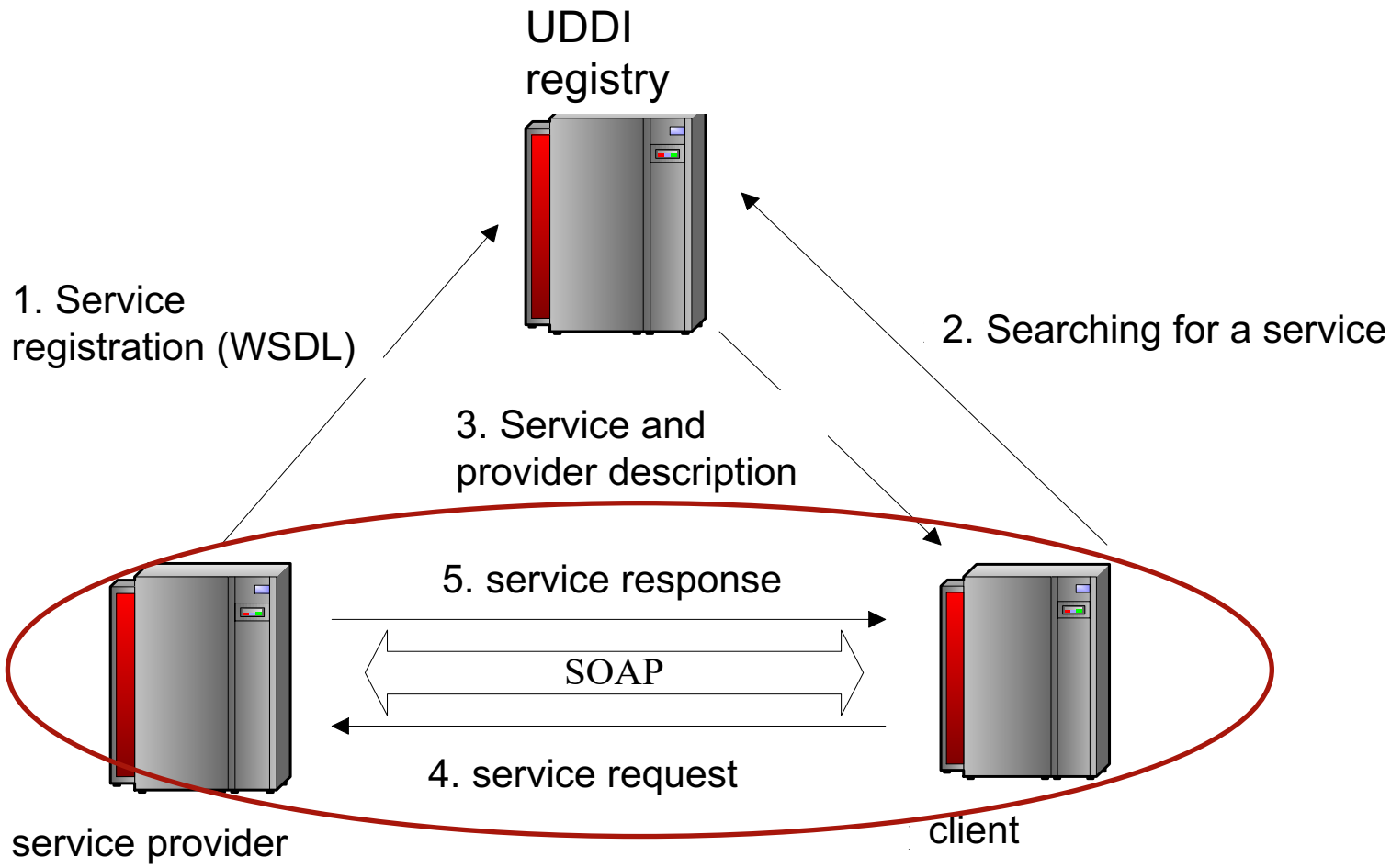
What is a Web service?

- Definition form W3C:
  - a software system identified by a URI
  - public interfaces and bindings are defined and described using XML
  - its definition can be discovered by other software systems
  - other systems may then interact with the Web service in a manner prescribed by its definition, using XML based messages conveyed by Internet protocols
- Intended to support machine-to-machine interactions over the network using messages
- Basic ideas is to build a platform and programming language-independent distributed invocation system out of existing Web standard
- Very loosely defined, when compared to CORBA, RMI, etc
- In comparison to the other RPC (Corba, RMI)
  - no problems with firewalls (HTTP) and less problems with security (HTTPS)
- XML-based distributed services system, with XML based protoclos
  - SOAP, WSDL, UDDI

WebService XML protocols (1)

- UDDI: Universal Description, Discovery and Integration
  - registry of business web services
  - allows to find needed web service on then contact with them
- SOAP: Simple Object Access Protocol
  - XML Message format between client and service
- WSDL: Web Service Description Language
  - describes how the service is to be used
  - like java Interface.
  - guideline for constructing SOAP messages.
  - WSDL is an XML language for writing APIs
- UDDI is now in a rare use
  - IBM, Microsoft, and SAP closed their public UDDI nodes in 2006
- In practice one doesn't need to work directly with SAOP and WSDL
  - there are many tools that generate the WSDL for you from class
  - SOAP messages are constructed automaticaly

WebService XML protocols (2)



SOAP request example

```
<?xml version='1.0' encoding='UTF-8'?>
<SOAP-ENV:Envelope xmlns:SOAP-
   ENV="http://schemas.xmlsoap.org/soap/envelope/"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xmlns:xsd="http://www.w3.org/2001/XMLSchema">
<SOAP-ENV:Body>
<ns1:add xmlns:ns1="urn:Calculator"
SOAP-
   ENV:encodingStyle="http://schemas.xmlsoap.org/soap/encoding/">
<a xsi:type="xsd:float">2.5</a>
<b xsi:type="xsd:float">3.5</b>
</ns1:add>
</SOAP-ENV:Body>
</SOAP-ENV:Envelope>
```

SOAP response example

```
HTTP/1.1 200 OK
Content-Type: text/xml; charset=utf-8
Content-Length: 446
Date: Sun, 15 Dec 2010 23:15:58 GMT
Server: Apache Tomcat/6 (HTTP/1.1 Connector)
<?xml version='1.0' encoding='UTF-8'?>
<SOAP-ENV:Envelope xmlns:SOAP-
   ENV="http://schemas.xmlsoap.org/soap/envelope/"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xmlns:xsd="http://www.w3.org/2001/XMLSchema">
<SOAP-ENV:Body>
<ns1:addResponse xmlns:ns1="urn:Calculator"
SOAP-
   ENV:encodingStyle="http://schemas.xmlsoap.org/soap/encoding/">
<return xsi:type="xsd:float">6.0</return>
</ns1:addResponse>
</SOAP-ENV:Body>
</SOAP-ENV:Envelope>
```

SOAP

- **S**imple **O**bject **A**ccess **P**rotocol
- Format for sending messages over Internet between programs
- XML-based
- Platform and language independent
- Simple and extensible
- Stateless, one-way
  - But applications can create more complex interaction patterns

WSDL

```
<xs:complexType name="MathInput">
            <xs:sequence>
               <xs:element name="x" type="xs:double"/>
               <xs:element name="y" type="xs:double"/>
            </xs:sequence> ...
<xs:element name="Add" type="MathInput"/>
<xs:element name="AddResponse" type="MathOutput"/> …
<message name="AddMessage">
      <part name="parameters" element="ns:Add"/>
</message> …
<portType name="MathInterface">
<operation name="Add">
         <input message="y:AddMessage"/>
         <output message="y:AddResponseMessage"/>
</operation>
```

Use of WSDL Documents

- Description of service interfaces:
  - Compile-time:
    - Developer uses WSDL before service deployment.
  - Run-time:
    - Client downloads WSDL description and uses the information it provides to execute the service.
- As a side-effect:
  - Developers can use WSDL to speed up the development of code,
  - WSDL→Java code,
  - Java interfaces → WSDL.

WebService conclusion

- Advantages
  - very simple to deploy
    - server and client
  - no problems with firewall (HTTP)
  - security (HTTPS)
  - easy way of communication in the heterogonous environment
    - similar approach in .Net
- Disadvantages
  - large size of messages (SOAP in XML)
  - “heavy” servers
  - Not true today (for example JAX-WS and Java SE)

REST

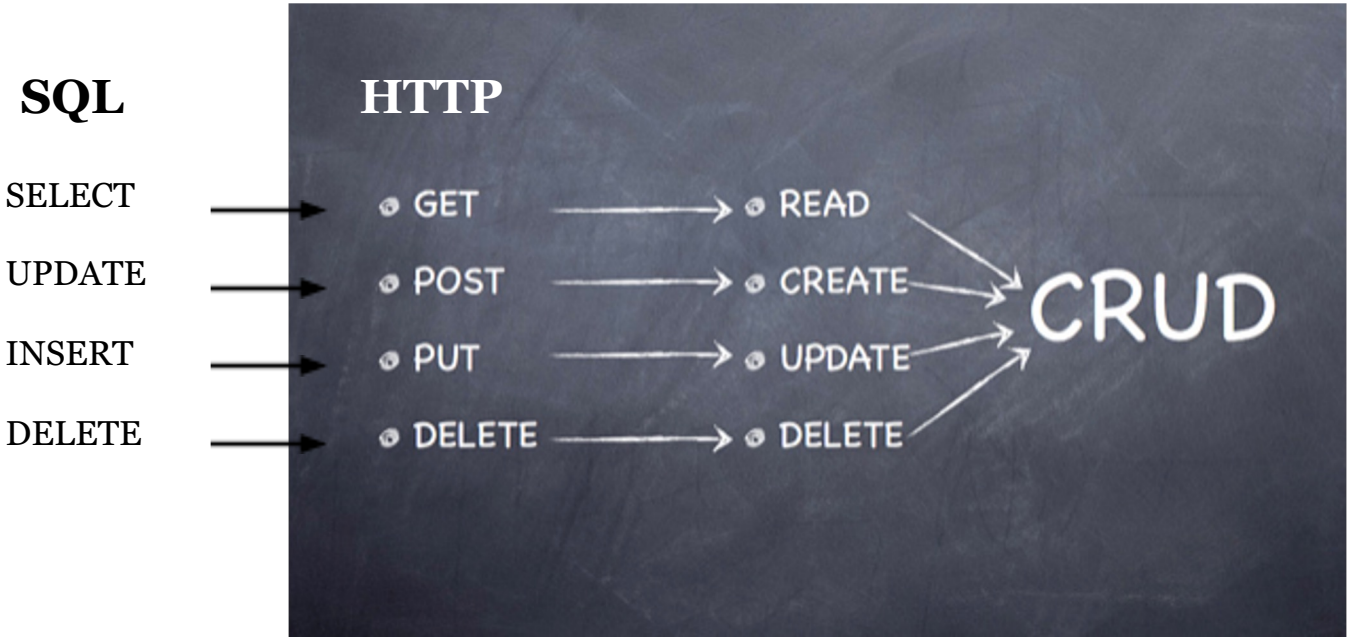
- **Re**presentational **S**tate **T**ransfer
- Architectural style (technically not a standard)
- Idea: a network of web pages where the client progresses through an application by selecting links
- When client traverses link, accesses new resource (i.e., transfers state)
- Uses existing standards, e.g., HTTP

REST Characteristics

- Client-Server: Clients pull representations
- Stateless: each request from client to server must contain all needed information.
- Uniform interface: all resources are accessed with a generic interface (HTTP-based)
- Interconnected resource representations
- Links:
  - aliexpress.com/category/women's-clothing
- Layered components - intermediaries, such as proxy servers, cache servers, to improve performance, security

1 - REST commands

REST



http://www.andyhoffman.net/2010/02/excel-services-needs-a-rest/

REST Principle

Everything is a RESOURCE

Examples:

- Customers
- Locations
- Items

Second REST Principle

Resources have NAMES

Examples:

- http://acme.com/customers/coyote
- http://world.com/usa/california/san_diego
- xri://@acme*warehouse/(+rockets)

Third RSET Principle

Resources support simple VERBS

Examples:

- Create, Read, Update, Delete
- POST, GET, PUT, DELETE

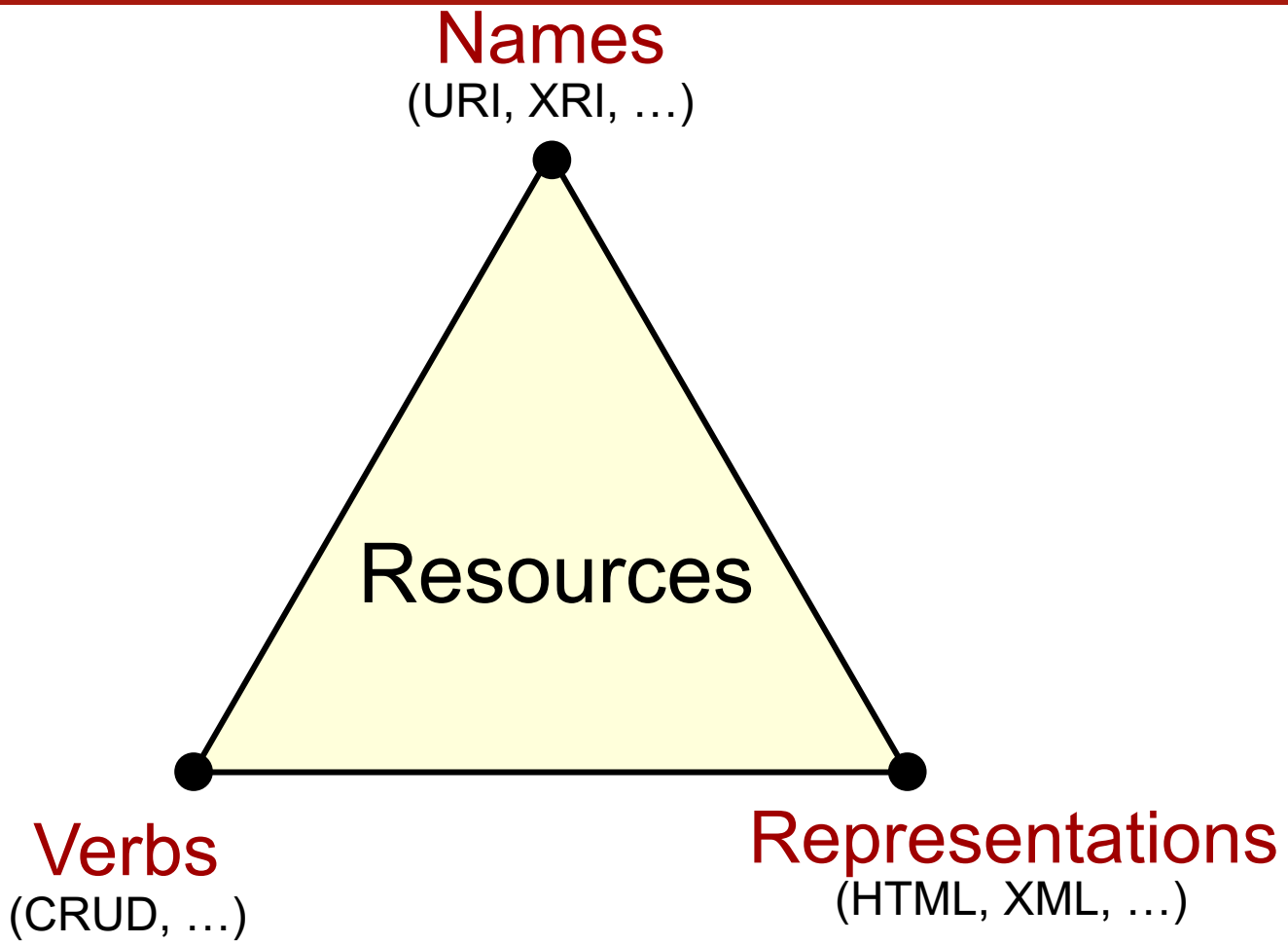
Fourth REST Principle

Resources have REPRESENTATIONS

Examples:

- HTML: text/html, application/xhtml+xml, …
- XML: text/xml, application/atom+xml, …
- Binary: image/png, application/pdf, …

REST in One Slide



REST Conclusions

- GET and POST used mostly, PUT and delete are rare
- Pure REST is rarely used. Now single GET or POST returns e.g. JSON with complex structure

XML vs JSON

JSON

```
{     "firstName": "John",
      "lastName": "Smith",
      "age": 25,
      "address": {
      "streetAddress":"21 2nd Street",
          "city": "New York",
          "state": "NY",
          "postalCode": "10021"
      },
      "phoneNumber": [
          { "type": "home", "number":
                "212 555- 1234" },
          { "type": "fax", "number":
               "646 555-4567" }
      ]
```

XML

```
<Person>
  <firstName>John</firstName>
  <lastName>Smith</lastName>
  <age>25</age>
  <address>
    <streetAddress>21 2nd Street
    </streetAddress>
    <city>New York</city>
    <state>NY</state>
    <postalCode>10021</postalCode>
  </address>
  <phoneNumber type="home">212 555 1234
  </phoneNumber>
  <phoneNumber type="fax">646 555-4567
  </phoneNumber>
</Person>
```

- Shorter then XML
- JavaScript based
- JSON schema
  - Was not used but it changed

XML Schema = validators, tools and ..

- Mapping (code generators)
  - XML ⇔ objects
  - XML Schema ⇔ types (classes)
- Large number of tools (DOM, SAX, JA
- Memory/CPU consumption

Summary: SOAP vs. REST

- communication protocol
  - Both: HTTP/HTTPS
  - endpoints
    - SOAP: path + SOAP
    - REST: path, methods, Mime type
  - arguments
    - SOAP: XML
    - REST: path, ?, POST
- data exchange format
  - SOAP: XML/XML Schema
  - REST: JSON/anything
- IDL – interface exchange
  - SOAP: WSDL – XML Schema
  - REST: nothing but openapi

Implementaion

- Easy everywhere
  - client wget, XMLHTTPRequest
- Frameworks:
  - JAVA: JAX-RS
  - Python:
    - flask, cherrypy, aiohttp
    - fastAPI

REST style in Java (JAX-RS)

This is the way to GET Cusomer with id 1 (GET /customers/{id}):

http://localhost:8080/RestApp/customers/1

From Class:

```
@Path("/customers/")
public class CustomersResource {
...
/** Returns a dynamic instance of CustomerResource used for entity navigation.
@return an instance of CustomerResource */
        @Path("{customerId}/")
        public CustomerResource
        getCustomerResource(@PathParam("customerId") Integer id) {
                CustomerResource resource = resourceContext.getResource(CustomerResource.class);
                resource.setId(id);
                return resource;
        }
}
```

FastAPI

```
app = FastAPI()

@app.get("/")
async def root():
        return {"message": "It works!"}

@app.get("/items/{item_id}")
async def read_item(item_id):
        return {"item_id": item_id}

@app.post("/lpmn/")
async def start_lpmn( request: Request, background_tasks:
BackgroundTasks, task: TaskLPMN, user: str =
Depends(get_token_header) ):
```

Problems with REST

- Lack of WSDL
  - Automatic generation of clients:
    - in homogenous env. it is not needed since we could have interfaces
    - We can say (it is not true) that this is not nedeed for script like languages
  - Checking of input data correctness
  - Automatic documentation
- Solutions in in REST:
  - RAML, Swagger based on JSON Schema

JSON Swagger / OpenAPI

- This is really the API spec, not documentation
- But it is essential for describing the API
- Can be used as input for API Documentation tools
- API authors should make use of Summary and Description fields!
- EXAMPLE
- Problems: have to be defined by humans


```
"paths": {
  "/pets": {
    "get": {
      "summary": "List all pets",
      "operationId": "listPets",
      "tags": [
        "pets"
      ],
      "parameters": [
        {
          "name": "limit",
          "in": "query",
          "description": "How many items to return at one time (max 100)",
          "required": false,
          "type": "integer",
          "format": "int32"
        }
      ],
      "responses": {
        "200": {
          "description": "An paged array of pets",
          "headers": {
            "x-next": {
              "type": "string",
              "description": "A link to the next page of responses"
            }
          },
          "schema": {
            "$ref": "#/definitions/Pets"
          }
        },
```

Other problems with classical REST

- `GET /businessPartner/12274526535/containers`

- Hard to specify and implement advanced requests with includes, excludes and especially with linked or nested resources
- Generic endpoints typically overfetch data
- Specific endpoints (per view) are getting pretty ugly to version hell
- **Over- or under-fetching of data is unavoidable**
- Too tight coupling between client (views) and server (endpoints)
- documentation often outdated
- developer uncertain about the responses (ambiguity, trial and error)

GraphQL

- From FaceBook ,2012,2015, 2018 => GraphQL foundation
- an open-source data query and manipulation language for APIs for fulfilling queries with existing data
- interprets strings from the client, and returns data in an understandable, predictable, pre-defined manner
- is an alternative, though not necessarily a replacement for REST
- not:
    - No Database
    - No Storage Engine
    - No Library
    - No Software to install (i.e. no GraphQL server to buy and install)
    - Not bound to a programming language
    - Not bound to a network transport protocol (can use HTTP, websockets, …)

GraphQL

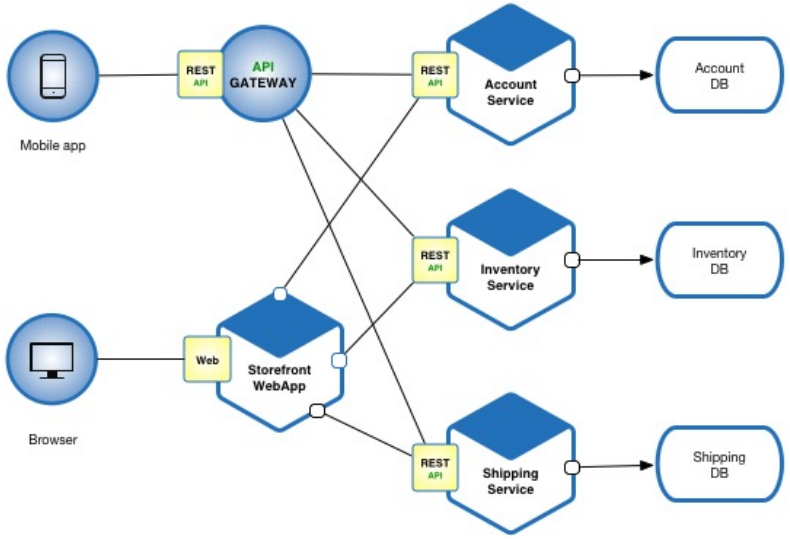
```
query threeLocations {
  atlanta: currentConditions(location: "30313") {
        hasPrecipitation
        temperatureF
        weatherText
        precipitationType
    }

  rochester: currentConditions(location: "55901") {
        hasPrecipitation
        temperatureF
        weatherText
        precipitationType
    }

  beverlyHills: currentConditions(location: "90210") {
        hasPrecipitation
        temperatureF
        weatherText
        precipitationType
    }
}
```

```
{
  "data": {
    "atlanta": {
      "hasPrecipitation": false,
      "temperatureF": 12.641566188496578,
      "weatherText": "Sunny",
      "precipitationType": null
    },
    "rochester": {
      "hasPrecipitation": true,
      "temperatureF": 11.727660249578708,
      "weatherText": "Overcast",
      "precipitationType": "SNOW"
    },
    "beverlyHills": {
      "hasPrecipitation": false,
      "temperatureF": 75.94192189884092,
      "weatherText": "Sunny",
      "precipitationType": null
    }
  }
}
```

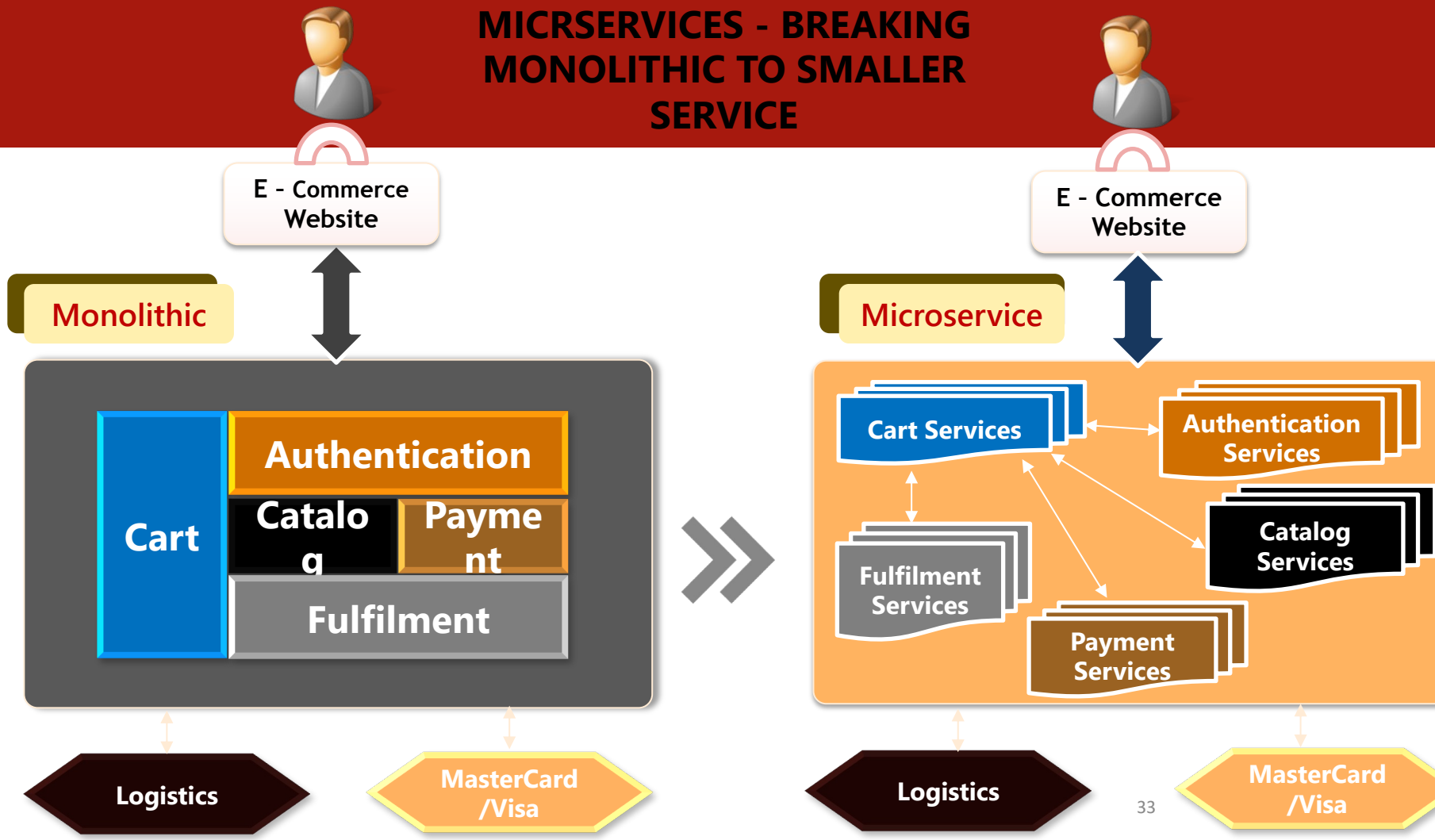
https://graphql.org/learn/

Micro-service

- Microservices is a variant of the service-oriented architecture (SOA) architectural style that structures an application as a collection of loosely coupled services.
- Microservices should be fine-grained and protocols should be lightweight.
- Application is easier to understand, develop and test.

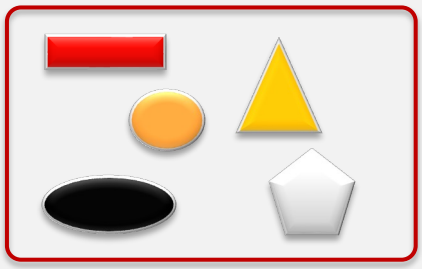
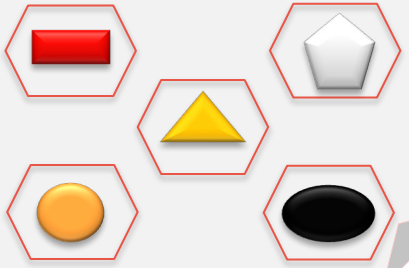
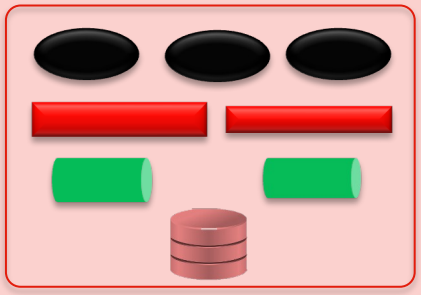
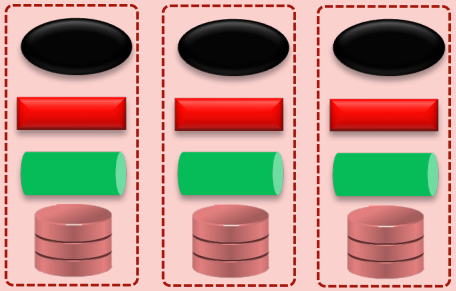


MICRSERVICES - BREAKING MONOLITHIC TO SMALLER SERVICE



From: qaistc.com/2016/wp-content/uploads/2016/12/Krishnachander_Kaliyaperumal.ppt

From: qaistc.com/2016/wp-content/uploads/2016/12/Krishnachander_Kaliyaperumal.ppt

	Monolithic	Microservice
Conceptual	One large application that serves the business function 	Small Independent autonomous, auto deployable, domain driven services of one application 
Technical	Each service is dependent on the other 	Each service has independent stack & de-centralized data 
Team		Embraces pure agile that is cross functional Every service team is small and responsible end to end.